

EXERCISE SOLUTIONS



Module 3: Advanced SQL Queries

Module 3 - 8 Hours

Advanced SQL Queries

Notes for the trainer

Core and stretch. The questions marked **(Core)** are the required set that fits the session. Debrief those with the whole room. Cover the unmarked stretch questions only if the batch worked ahead, or leave them as take-home reading.

Release after the practice window. Marks total 100.

Section	Questions	Marks
Level 1 - Understanding	Q1 to Q10	25
Level 2 - Applying SQL	Q11 to Q29	50
Level 3 - Analysis	Q30 to Q35	25

All expected outputs below are real results from the standard dataset.

Level 1 - Understanding

Q1 (Core)

Four join conditions. n tables need $n-1$ conditions.

Supplying three means one pair of tables is joined with no condition, producing a Cartesian product between them. Every row on one side is paired with every row on the other, so the result set multiplies. On this dataset that turns a few hundred rows into hundreds of thousands. On production volumes the query does not finish, and it consumes the server while failing.

The symptom to recognise: a row count that is a **product** of two table sizes rather than roughly the size of the largest table.

Q2 (Core)

The inner join returns one row per customer-account pair, so a customer with no account disappears and a customer with two accounts appears twice. The left join keeps every customer regardless, filling the account columns with NULL where there is no match.

On this dataset both return **36 rows**, because every customer happens to hold at least one account, so there is nothing for the left join to add. Learners who answer "the left join always returns more" have the reasoning wrong. The correct statement is that a left join returns **at least as many** rows as the equivalent inner join, and strictly more only when the left table has rows with no match.

Give the mark for "at least as many, never fewer". Deduct for "always more".

Q3 (Core)

The condition `l.status = 'OVERDUE'` sits in `WHERE` and refers to the **optional** table. For a branch with no loans, `l.status` is NULL, so the comparison is NULL rather than TRUE, and `WHERE` discards the row. The outer join is silently downgraded to an inner join.

```
SELECT b.branch_name, l.loan_id
FROM   branch b
      LEFT JOIN loan l ON l.branch_id = b.branch_id
                        AND l.status = 'OVERDUE'
ORDER BY b.branch_name;
```

Moving the condition into `ON` filters which loans are eligible to match, while still keeping every branch.

Q4

Shape: outer-join the second table onto the first, then keep only the rows where the join failed, by testing a column of the second table for NULL.

Which column: the second table's **primary key**. A primary key can never be NULL in a real row, so a NULL there can only mean "no row matched".

Why not a nullable column: testing something like `f.nominee_name IS NULL` would match two different populations - rows that did not join at all, and rows that joined perfectly well but happen to have no nominee recorded. The two get mixed together and the answer is wrong in a way that is very hard to see.

Q5

Because a Branch Manager's `manager_id` is NULL. An inner join requires a matching row in the manager alias, and NULL matches nothing, so every Branch Manager is dropped.

Concretely: `SELECT COUNT(*)` with an inner join returns **16**, with a left join **24**. The eight missing rows are the eight Branch Managers - the most senior people in the organisation, silently absent from the org chart.

Q6 (Core)

Expression	Counts NULLs	Value on <code>account</code>
<code>COUNT(*)</code>	Yes, counts every row	36
<code>COUNT(interest_rate)</code>	No, skips NULLs	36 (no NULLs in this column)
<code>COUNT(DISTINCT account_type)</code>	No	3 - SAVINGS, CURRENT, SALARY
<code>AVG(balance)</code>	No, divides by the non-null count	220372.35

The third answer is the interesting one. The check constraint permits four types including NRI, but the data contains only three. `GROUP BY` and `COUNT(DISTINCT)` **report what is in the data, never the full list of permitted values**. A report of "accounts by type" will silently omit NRI, and a business user will assume the bank has no NRI product rather than no NRI accounts.

Q7 (Core)

The rule: every column in the `SELECT` list must either appear inside an aggregate function or be named in the `GROUP BY` clause.

Why the engine cannot pick: the output has one row per group, but the group contains many rows. Branch 1 has four accounts with four different account numbers. There is no principled basis for choosing one of them, and any choice would be arbitrary and unstable between runs. Rather than return a plausible-looking wrong answer, the engine rejects the query. Older MySQL versions did pick, silently, which is why a great deal of legacy reporting SQL is quietly wrong.

Q8 (Core)

	WHERE	HAVING
Filters what	Individual rows	Whole groups
Runs when	Before <code>GROUP BY</code>	After <code>GROUP BY</code> and aggregation
Can reference aggregates	No	Yes
Which to prefer	<code>WHERE</code> , whenever the condition can be expressed on a single row, because fewer rows reach the grouping step and the query does less work	

Q9 (Core)

Six of the twenty deposits have a NULL `nominee_name`, so the subquery returns a list containing NULL. `NOT IN` expands into a chain of `<>` comparisons joined by `AND`, and any comparison with NULL yields NULL. `TRUE AND`

NULL is NULL, WHERE keeps only TRUE, so no row can ever pass. The result is zero rows - not fewer rows, none.

Fix 1 - exclude the NULLs from the subquery:

```
SELECT COUNT(*) FROM customer c
WHERE c.first_name NOT IN (SELECT nominee_name FROM fixed_deposit
                          WHERE nominee_name IS NOT NULL);
```

Fix 2 - use NOT EXISTS, which is NULL-safe:

```
SELECT COUNT(*) FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM fixed_deposit f WHERE f.nominee_name = c.first_name);
```

Fix 2 is preferred. It states the intent directly, it cannot be broken by a future NULL, and it does not depend on anyone remembering this rule.

Q10

Fan-out is the multiplication of rows that happens when a parent row joins to several child rows. One customer with two accounts and twenty-one transactions occupies twenty-one rows after joining all three tables.

A wrong but believable report: "total deposits and transaction value per branch". Joining `branch`, `customer`, `account` and `account_txn` and then writing `SUM(a.balance)` adds each account's balance once **per transaction on that account**. A branch with 900,000 in deposits and 60 transactions reports something in the tens of millions. It is believable because the number is large and the query looks correct, and it is caught only when somebody adds the account balances by hand.

The fix is either `COUNT(DISTINCT ...)` for counts, or aggregating each grain separately and joining the summaries.

Level 2 - Applying SQL

Q11 (Core)

```
SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number, k.card_type, k.card_number_masked, k.status
FROM   card k
       JOIN account a ON a.account_id = k.account_id
       JOIN customer c ON c.customer_id = a.customer_id
       JOIN branch b ON b.branch_id = a.branch_id
ORDER BY b.branch_name, customer_name;
```

32 rows, one per card. Four tables, three `ON` conditions.

Q12 (Core)

```
SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number, ROUND(a.balance, 2) AS balance
FROM   customer c
       JOIN branch b ON b.branch_id = c.home_branch_id
       LEFT JOIN account a ON a.customer_id = c.customer_id
WHERE  b.branch_name IN ('Koramangala', 'Whitefield')
ORDER BY a.balance DESC NULLS LAST;
```

The branch filter is in `WHERE` and the account is the optional table, so this is correct: `branch` is required, `account` is not. Putting the branch condition in the `LEFT JOIN`'s `ON` clause would be the mistake here.

Q13 (Core)

```
SELECT a.account_number, a.account_type, a.status,
       c.first_name || ' ' || c.last_name AS customer_name
FROM   account a
       JOIN customer c ON c.customer_id = a.customer_id
       LEFT JOIN account_txn t ON t.account_id = a.account_id
WHERE  t.txn_id IS NULL;
```

account_number	account_type	status	customer_name
ICIC0000009001	CURRENT	DORMANT	Aarav Sharma

One account. Note that `customer` is joined with an inner join and `account_txn` with a left join, in the same query. Each join gets the type the question requires.

Q14

```
SELECT a.account_number, a.account_type,
       c.first_name || ' ' || c.last_name AS customer_name
FROM   account a
       JOIN customer c ON c.customer_id = a.customer_id
```

```

LEFT JOIN card k ON k.account_id = a.account_id
WHERE k.card_id IS NULL
ORDER BY a.account_number;

```

account_number	account_type	customer_name
ICIC0000009033	SAVINGS	Pooja Rao
ICIC0000009034	CURRENT	Suresh Singh
ICIC0000009035	SALARY	Anjali Verma
ICIC0000009036	SAVINGS	Anjali Verma

Count: 4. These are the four most recently opened accounts, which is exactly the pattern you would expect from a card issuance backlog - and a useful operational finding rather than a data error.

Q15

```

SELECT e.first_name || ' ' || e.last_name AS employee,
       e.designation, b.branch_name,
       COALESCE(m.first_name || ' ' || m.last_name, '(no manager)') AS reports_to
FROM   employee e
       JOIN branch b ON b.branch_id = e.branch_id
       LEFT JOIN employee m ON m.employee_id = e.manager_id
ORDER BY b.branch_name, e.employee_id;

```

24 rows. `COALESCE` is needed because concatenating anything with NULL yields NULL, so without it the Branch Managers' `reports_to` would be blank rather than labelled.

Q16

```

SELECT m.first_name || ' ' || m.last_name AS manager,
       b.branch_name,
       COUNT(e.employee_id) AS team_size,
       ROUND(SUM(e.salary), 2) AS team_salary
FROM   employee m
       JOIN branch b ON b.branch_id = m.branch_id
       LEFT JOIN employee e ON e.manager_id = m.employee_id
WHERE  m.designation = 'Branch Manager'
GROUP BY m.employee_id, m.first_name, m.last_name, b.branch_name
ORDER BY b.branch_name;

```

Eight rows, each with `team_size` of 2. The manager's own salary is excluded automatically, because `e` holds only the people whose `manager_id` points at `m`, and a manager does not report to themselves. Grouping by `m.employee_id` as well as the name protects against two managers sharing a name.

Q17

```

CREATE TABLE loan_risk_band (
    band_code      VARCHAR(4)    NOT NULL,
    band_name      VARCHAR(20)   NOT NULL,
    min_outstanding NUMERIC(15,2) NOT NULL,
    max_outstanding NUMERIC(15,2) NOT NULL,
    CONSTRAINT pk_loan_risk_band PRIMARY KEY (band_code)
);

```

```

INSERT INTO loan_risk_band VALUES
  ('L1','Small',      0.00,   500000.00),
  ('L2','Medium',    500000.00, 2000000.00),
  ('L3','Large',     2000000.00, 5000000.00),
  ('L4','Very Large', 5000000.00, 9999999.00);

SELECT r.band_name,
       COUNT(*)                AS loans,
       ROUND(SUM(l.outstanding_amount), 2) AS total_outstanding
FROM   loan l
       JOIN loan_risk_band r ON l.outstanding_amount >= r.min_outstanding
                          AND l.outstanding_amount < r.max_outstanding
GROUP BY r.band_code, r.band_name
ORDER BY r.band_code;

```

The bands are half-open: `>= min AND < max`. Using `BETWEEN` on both would make the boundaries overlap, and a loan sitting exactly on a boundary would be counted in two bands, inflating both the count and the total. Grouping by `band_code` as well as `band_name` keeps the sort in band order rather than alphabetical.

Q18 (Core)

```

SELECT b.branch_name,
       COUNT(DISTINCT c.customer_id) AS customers,
       COUNT(a.account_id)           AS accounts,
       ROUND(COALESCE(SUM(a.balance), 0), 2) AS total_balance,
       ROUND(AVG(a.balance), 2)           AS avg_balance
FROM   branch b
       LEFT JOIN customer c ON c.home_branch_id = b.branch_id
       LEFT JOIN account a ON a.customer_id    = c.customer_id
GROUP BY b.branch_name
ORDER BY total_balance DESC;

```

`COUNT(DISTINCT c.customer_id)` is required because a customer with two accounts otherwise counts twice. `COUNT(a.account_id)` rather than `COUNT(*)` so a branch with no accounts reports 0 rather than 1. Every branch has customers in this dataset, so no zero rows appear, but the query is correct if one did.

Q19

```

SELECT l.loan_type,
       COUNT(*)                AS loans,
       ROUND(SUM(l.principal_amount), 2) AS principal,
       ROUND(SUM(l.outstanding_amount), 2) AS outstanding,
       ROUND(100.0 * SUM(l.outstanding_amount) / SUM(l.principal_amount), 1) AS pct_outstanding
FROM   loan l
GROUP BY l.loan_type
ORDER BY outstanding DESC;

```

loan_type	loans	principal	outstanding	pct_outstanding
HOME	3	6200000.00	5558681.93	89.7
GOLD	2	6080000.00	3254891.93	53.5
PERSONAL	2	3450000.00	2751096.02	79.7
AUTO	3	2250000.00	1733612.66	77.0

```
EDUCATION |      2 | 1750000.00 | 1319520.49 |      75.4
```

Note `SUM(outstanding) / SUM(principal)`, **not** `AVG(outstanding / principal)`. The first is the portfolio's true position; the second is the unweighted average of individual ratios, which lets a tiny loan influence the result as much as a large one. Confusing the two is a classic error in credit reporting.

The `100.0` rather than `100` forces decimal arithmetic. With integer division on some engines, every result would be 0.

Q20 (Core)

```
SELECT t.channel,
       SUM(CASE WHEN t.status = 'SUCCESS' THEN 1 ELSE 0 END) AS successful,
       SUM(CASE WHEN t.status = 'FAILED' THEN 1 ELSE 0 END) AS failed,
       ROUND(SUM(CASE WHEN t.status = 'SUCCESS' THEN t.amount ELSE 0 END), 2) AS success_value,
       ROUND(100.0 * SUM(CASE WHEN t.status = 'FAILED' THEN 1 ELSE 0 END) / COUNT(*), 2) AS failure_rate
FROM   account_txn t
GROUP BY t.channel
ORDER BY failure_rate DESC;
```

channel	successful	failed	failure_rate
POS	56	4	6.67
NEFT	61	4	6.15
IMPS	52	3	5.45
ONLINE	57	2	3.39
ATM	61	2	3.17
BRANCH	57	1	1.72
UPI	64	0	0.00
RTGS	71	0	0.00

Sixteen failures in total. Point out that with only 60 transactions per channel, a difference between 6.67 and 5.45 percent is one transaction and means nothing. **Rates computed over small denominators are noise**, and a report that ranks them invites the reader to act on randomness. In production these denominators are in the millions and the ranking is meaningful.

Q21

```
SELECT b.branch_name, COUNT(*) AS loans,
       ROUND(SUM(l.outstanding_amount), 2) AS total_outstanding
FROM   loan l
       JOIN branch b ON b.branch_id = l.branch_id
GROUP BY b.branch_name
HAVING SUM(l.outstanding_amount) > 2000000
ORDER BY total_outstanding DESC;
```

branch_name	loans	total_outstanding
T Nagar Chennai	3	6215460.74
Koramangala	3	4780879.96
Fort Mumbai Main	3	2655077.80

Q22 (Core)

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       COUNT(*) AS accounts,
       ROUND(SUM(a.balance), 2) AS combined_balance
FROM   customer c
       JOIN account a ON a.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(*) >= 2
ORDER BY combined_balance DESC;
```

customer_id	customer_name	accounts	combined_balance
1020	Ritu Banerjee	2	654237.30
1030	Anjali Verma	2	587663.55
1010	Priya Verma	2	453005.40
1025	Farhan Gupta	2	437520.63
1015	Imran Bose	2	187104.39
1005	Karthik Gupta	2	119366.88

Six customers hold two accounts each. `HAVING COUNT(*) >= 2` cannot be written in `WHERE`, because the count does not exist until the rows are grouped.

Q23

```
SELECT TO_CHAR(t.txn_date, 'YYYY-MM') AS month,
       COUNT(*) AS txns,
       ROUND(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END), 2) AS credits,
       ROUND(SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END), 2) AS debits,
       ROUND(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE -t.amount END), 2) AS net
FROM   account_txn t
WHERE  t.status = 'SUCCESS'
GROUP BY TO_CHAR(t.txn_date, 'YYYY-MM')
ORDER BY month;
```

month	txns	credits	debits
2026-01	176	3441051.86	5206131.14
2026-02	135	3194498.15	4911830.04
2026-03	168	3724339.24	5117754.77

Debits exceed credits every month, which is normal for a retail transaction table where salary credits arrive as a few large amounts and spending leaves as many small ones.

On SQL Server use `FORMAT(txn_date, 'yyyy-MM')` or `CONVERT(char(7), txn_date, 126)`; on MySQL use `DATE_FORMAT(txn_date, '%Y-%m')`. Grouping by a formatted string is convenient but prevents index use, which Module 5 addresses.

Q24

```
SELECT account_number, ROUND(balance, 2) AS balance
FROM   account
WHERE  balance > 2 * (SELECT AVG(balance) FROM account)
ORDER BY balance DESC;
```

One account qualifies. The average is 220,372.35, so the threshold is 440,744.70, and only the largest balance of 473,345.84 clears it. A learner reporting several has probably compared against the average of a filtered subset.

Q25

```
SELECT b.branch_name,
       (SELECT ROUND(MAX(a.balance), 2)
        FROM   account a
        WHERE  a.branch_id = b.branch_id) AS largest_account
FROM   branch b
ORDER BY largest_account DESC;
```

branch_name	largest_account
Connaught Place	473345.84
Andheri East	439665.56
Whitefield	421425.91
Koramangala	410879.83
Fort Mumbai Main	408783.99
Banjara Hills	400457.72
T Nagar Chennai	276393.25
Baner Pune	119212.62

The subquery references `b.branch_id` from the outer query, so it is correlated and conceptually runs once per branch. The equivalent `GROUP BY` version is cheaper on large data; accept either, but note which is which.

Q26

```
-- NOT EXISTS
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
WHERE  EXISTS (SELECT 1 FROM loan l          WHERE l.customer_id = c.customer_id)
       AND NOT EXISTS (SELECT 1 FROM fixed_deposit f WHERE f.customer_id = c.customer_id)
ORDER BY c.customer_id;

-- LEFT JOIN anti-join
SELECT DISTINCT c.customer_id, c.first_name, c.last_name
FROM   customer c
       JOIN loan l ON l.customer_id = c.customer_id
       LEFT JOIN fixed_deposit f ON f.customer_id = c.customer_id
WHERE  f.deposit_id IS NULL
ORDER BY c.customer_id;
```

Both return the same **5 customers**: 1009, 1011, 1013, 1017 and 1019.

Note the `DISTINCT` in the second version. A customer with two loans would otherwise appear twice, because the join to `loan` fans out. The `NOT EXISTS` version needs no `DISTINCT`, which is one reason to prefer it.

Q27

```
SELECT ct.customer_id, ct.customer_name, ct.total_value
FROM (SELECT c.customer_id,
             c.first_name || ' ' || c.last_name AS customer_name,
             SUM(t.amount) AS total_value
      FROM customer c
           JOIN account a ON a.customer_id = c.customer_id
           JOIN account_txn t ON t.account_id = a.account_id
      WHERE t.status = 'SUCCESS'
      GROUP BY c.customer_id, c.first_name, c.last_name) AS ct
WHERE ct.total_value > (SELECT AVG(total_value)
                       FROM (SELECT SUM(t.amount) AS total_value
                             FROM account_txn t
                             JOIN account a ON a.account_id = t.account_id
                             WHERE t.status = 'SUCCESS'
                             GROUP BY a.customer_id) AS per_customer)
ORDER BY ct.total_value DESC;
```

The inner aggregate has to be computed twice, once to list and once to average. Module 6 replaces this with a single common table expression referenced twice, which is both clearer and cheaper.

Q28

```
SELECT customer_id, product_type, ROUND(amount, 2) AS amount
FROM (SELECT customer_id, 'DEPOSIT' AS product_type, principal_amount AS amount FROM fixed_deposit
      UNION ALL
      SELECT customer_id, 'LOAN', outstanding_amount FROM loan
      UNION ALL
      SELECT customer_id, 'ACCOUNT', balance FROM account) AS holdings
ORDER BY customer_id, product_type;
```

68 rows: 20 deposits, 12 loans, 36 accounts. **UNION ALL** is correct here - de-duplicating would merge two accounts that happen to hold identical balances, silently losing a real holding.

Q29 (Core)

```
SELECT customer_id FROM fixed_deposit INTERSECT SELECT customer_id FROM loan; -- 7
SELECT customer_id FROM fixed_deposit EXCEPT SELECT customer_id FROM loan; -- 13
SELECT customer_id FROM loan EXCEPT SELECT customer_id FROM fixed_deposit; -- 5
SELECT customer_id FROM loan UNION SELECT customer_id FROM fixed_deposit; -- 25
```

Group	Count
Both a deposit and a loan	7
Deposit but no loan	13
Loan but no deposit	5
At least one of the two	25

The consistency check: $7 + 13 + 5 = 25$, which matches the union. Also $7 + 13 = 20$ distinct deposit holders and $7 + 5 = 12$ distinct loan holders, both correct. Five of the thirty customers hold neither product.

Insisting on this arithmetic is the habit to build. Any set-based analysis that does not add up has a bug, usually a NULL or a duplicate.

Level 3 - Analysis and decisions

Q30 (Core)

Four defects.

1. **Fan-out on the customer count.** `COUNT(c.customer_id)` counts one row per transaction, not per customer. A customer with 21 transactions is counted 21 times.
2. **Fan-out on the balance.** `SUM(a.balance)` adds each account's balance once per transaction on that account. The total is inflated by roughly the average transaction count, which is around fourteen here. This is the number that "looks a bit high".
3. **BETWEEN on a timestamp.** `txn_date` is a `TIMESTAMP`, so the range ends at `2026-03-31 00:00:00` and the whole of 31 March is missing.
4. **Inner joins where outer joins are needed.** A branch with no transactions in the window disappears from the report entirely, rather than appearing with zeros. Also, `t.status` is not filtered, so failed transactions are counted as business.

Corrected version. The two grains are aggregated separately and then joined, which is the general fix for fan-out.

```
WITH deposits AS (
    SELECT c.home_branch_id AS branch_id,
           COUNT(DISTINCT c.customer_id) AS customers,
           COUNT(a.account_id) AS accounts,
           SUM(a.balance) AS total_deposits
    FROM   customer c
           LEFT JOIN account a ON a.customer_id = c.customer_id
    GROUP BY c.home_branch_id
),
txns AS (
    SELECT a.branch_id, SUM(t.amount) AS total_transacted
    FROM   account_txn t
           JOIN account a ON a.account_id = t.account_id
    WHERE  t.txn_date >= DATE '2026-01-01'
           AND t.txn_date < DATE '2026-04-01'
           AND t.status = 'SUCCESS'
    GROUP BY a.branch_id
)
SELECT b.branch_name,
       COALESCE(d.customers, 0) AS customers,
       COALESCE(d.accounts, 0) AS accounts,
       ROUND(COALESCE(d.total_deposits, 0), 2) AS total_deposits,
       ROUND(COALESCE(x.total_transacted, 0), 2) AS total_transacted
FROM   branch b
       LEFT JOIN deposits d ON d.branch_id = b.branch_id
       LEFT JOIN txns x ON x.branch_id = b.branch_id
ORDER BY total_deposits DESC;
```

Marking: 2 marks per defect identified, 2 for a rewrite that separates the grains. Accept `COUNT(DISTINCT)` patched onto the original for partial credit, but point out that it fixes the count and not the sum.

Q31

`GROUP BY` produces **one row per group**. "Top three per branch" needs three rows per group, and it needs each account ranked **within** its branch. `GROUP BY` has no concept of position inside a group, and `HAVING` only filters whole groups, so neither can express it.

What is needed is a **window function**: `ROW_NUMBER() OVER (PARTITION BY branch_id ORDER BY balance DESC)`, which numbers the rows inside each branch without collapsing them, and then filtering on that number.

That is **Module 6**, which covers window functions, common table expressions and recursive queries.

A learner may propose a correlated subquery counting how many accounts in the same branch have a higher balance. That works, it is the classic pre-window-function idiom, and it is worth full marks with a note that it re-scans the table once per row.

Q32 (Core)

Model answer.

A report whose correct answer is usually zero gives you no feedback when it breaks, because a broken report and a healthy bank look identical. Six weeks of silence was interpreted as six weeks of compliance, when it was in fact six weeks of no monitoring at all.

The design lesson is that every such check needs a **positive control**: something that proves the query still runs and still finds things. In practice that means reporting the size of the population examined alongside the exceptions found, so "0 exceptions from 412,000 transactions checked" is distinguishable from "0 exceptions from 0 transactions checked". A deliberately seeded known-bad row that the report must always find is even stronger, as is alerting when the population count itself drops unexpectedly.

More generally: prefer `NOT EXISTS` over `NOT IN` so this particular failure cannot occur, and treat any exception report that has never fired as untested rather than as clean.

Q33

Without changing the SQL:

1. **Index the access paths the query already uses**, particularly `account_txn(account_id, txn_date)` and the foreign key columns joined on. An index the planner can use converts repeated table scans into targeted lookups. **Module 5**.
2. **Move the report off the transactional primary onto a read replica**, so a 40-second scan no longer competes with branch opening traffic for buffer cache and I/O. **Module 9**.
3. **Materialise the pack overnight** rather than computing it live at 9 a.m. A scheduled job writes the results to a summary table, and the morning query reads a few dozen pre-computed rows. **Modules 10 and 11** cover the job scheduling and the ETL that populates it.

Also creditable: caching the result in Redis with a suitable expiry, since the pack does not change during the day (**Module 7**); and partitioning `account_txn` by month so a quarter's report touches three partitions instead of the whole table (**Module 5**).

Marking: 2 marks per action with the correct module named. Deduct if the learner rewrites the SQL - the question explicitly forbade it, and the point is that performance is not only a query-writing problem.

Q34

Approach. Extract the same columns from both systems, into two tables with identical structure, and run `EXCEPT` in both directions.

```
-- In the acquired bank's extract but not in ICICI core: rows that failed to load
SELECT txn_id, account_id, txn_date, amount FROM acquired_extract
EXCEPT
SELECT txn_id, account_id, txn_date, amount FROM icci_core_txn;

-- In ICICI core but not in the extract: rows that should not be there
SELECT txn_id, account_id, txn_date, amount FROM icci_core_txn
EXCEPT
SELECT txn_id, account_id, txn_date, amount FROM acquired_extract;
```

What a non-empty result means. In the first direction, transactions the acquired bank holds that never arrived - a load failure, a rejected row, or a filter applied by mistake. In the second, transactions ICICI now holds that the source never had - duplicates from a re-run, rows from a different date range, or a join in the load that fanned out.

A difference that is not a missing row. A value mismatch. If `amount` was 1234.50 in the source and loaded as 1234.5 into a column with different scale, or a `TIMESTAMP` lost its time component moving into a `DATE` column, the row appears in **both** directions of the `EXCEPT` - once as missing and once as extra - even though nothing is lost. This is why a non-empty result must be investigated by key before it is reported as data loss, and why row counts alone are never sufficient evidence that a migration succeeded.

Q35

Model answer.

Ms Rao,

The branch performance pack is now a single query set against the core banking database, so the numbers in it come from one source and will agree with each other. It covers five sections: the branch summary, dormant customers, the ten largest transactions, deposits maturing shortly, and product holdings.

Two things stand out this quarter. T Nagar Chennai took in 2,691,432 against Whitefield's 347,122, a gap of nearly eight to one on comparable customer numbers, which is worth understanding before the next target-setting round. Separately, seven of the eight branches paid out more than they took in over the quarter; only Banjara Hills was close to balanced. That may be seasonal, but it is the first thing I would ask the branch heads about.

One limitation you should know about. The pack reports what is recorded against each customer's **home branch**, not where the transaction physically took place. A Fort Mumbai customer withdrawing cash in Pune is counted against Fort Mumbai. For a branch footfall discussion that is the wrong attribution, and we would need to capture the transacting branch on each transaction to answer it properly. I would rather tell you that now than have the numbers used for a decision they cannot support.

Marking: 5 marks. Look for two findings genuinely drawn from the data with figures attached, a real limitation stated honestly, and a tone that a business reader would act on. Deduct for findings that are not in the data, and for limitations that are generic rather than specific to this schema.